In the previous chapter, we gave an agent a difficult algorithmic problem and a lot of autonomy. It researched strategies, tried several approaches, got stuck, changed direction, and eventually found diagonal layering.

But circle packing had one enormous advantage that I did not appreciate enough at the beginning: we knew exactly what good meant.

There was an Immutable Harness. Run the program and you got a number. Circles overlapped or they did not; the score improved or it did not. The agent could spend an hour pursuing some bizarre geometric idea and I did not have to sit beside it wondering whether version seventeen had more soul. We ran the evaluator.

Most of the things I actually want AI to help me with are not like that.

“Is this explanation pedagogically effective?” does not have a unit test. “Would a confused student understand this visualization?” cannot be settled with an `assert`. Two competent people can look at the same design, disagree completely, then switch sides five minutes later after using it. The feedback is subjective, noisy, sometimes contradictory, and often becomes clearer only after you have built the thing you were supposedly trying to specify beforehand.

I picked educational demos for Merge Sort and Count-Min Sketch because they were still bounded—you can actually finish one before civilization collapses—but they live on the messier side of the boundary. You have to decide what to explain, what to leave out, how the interaction should work, how much should be visible at once, and what another person is likely to understand from any of it.

The ambition was intentionally high. I wanted something closer to the best Distill articles or Jay Alammar’s visual explanations than to the usual “here are some bars moving around; congratulations, you have learned sorting.” The algorithm itself is usually the easy part. The difficult part is deciding what to show, when to show it, and what representation might make an idea suddenly click.

Circle packing let the search be complicated because judgment was simple.

Here judgment had become part of the problem.

The problem-solving layer I eventually started calling **Deep Mode** grew out of one question: could the system take over some of the work of deciding what to try next?

Not just implementation. The inquiry itself. Build another version?

Research the failure? Retrieve an old idea? Split into independent branches? Change perspective? Abandon the direction?

Before trying to automate that, I had to notice how much of the work around the model had already moved into the machine.

How We Got Here

Coding first appeared as a strange side effect of language modeling.

The earliest useful tasks were conveniently small. Give the model a function signature, a comment, or a programming problem and ask it to fill in the implementation. Benchmarks such as HumanEval and APPS made this measurable: could a model turn a specification into a program that survived tests?

Then tools such as GitHub Copilot put that capability inside the editor. Instead of asking a chatbot for code and carrying the answer back yourself, you could describe what should happen next and watch code appear underneath it.

This was useful enough that the limitations became interesting.

A real software task rarely arrives as an isolated function with a docstring politely explaining what needs to change. Someone says invoices occasionally show the wrong tax after a refund. Somewhere inside a 150,000-line CRM there is a reason. It may involve a controller, a database model, an old helper function, a test written three years ago, and an API whose behavior everyone on the team knows but nobody thought to document.

By the time GPT-4 arrived, I increasingly wanted to use models on exactly these problems. The workflow was ridiculous. Find the file you suspect, copy a class into ChatGPT, describe the bug, copy the suggested patch into the editor, run the program, discover a new error, copy the traceback, paste that back into ChatGPT, repeat.

My first agent-computer interface was copy and paste.

The model might be doing sophisticated reasoning in the middle, but I performed every interaction with the software around it. I searched the repository, decided which file mattered, assembled the context, applied the edit, ran the tests, and carried back whatever reality had said about the edit.

COPY-PASTE · CORNER-BLEED

Corner scene: a person carrying an armful of paper strips (code fragments) between two desks — one with an old terminal, one with a glowing chat window — a worn path visible in the floorboards between them. Weary, funny, tender.

Then the bug crossed three files and context itself became a job. Paste one class but forget the interface it implements; the model confidently invents a method that does not exist. Add the interface and now it needs the database schema. Add the schema and another helper suddenly matters. Eventually half the repository is sitting in the conversation and somehow the model understands less.

A lot of early LLM programming consisted of building a tiny artificial universe around the model: here is the relevant class; here is the schema; ignore these twelve methods; this innocent-looking helper controls payments, so please do not touch it unless you enjoy incident calls.

We learned an obvious lesson surprisingly slowly: more context and better context are different things. If somebody asks for a spoon, emptying the entire kitchen onto the table does not necessarily help.

Software-engineering benchmarks exposed the same gap. SWE-bench changed the unit of evaluation. Its tasks came from real GitHub issues. Now a system had to work inside an existing repository, locate the relevant code, understand relationships across files, make an appropriate change and survive the tests.

Eventually we stopped carrying the loop by hand.

Give the model access to the repository. Let it search for symbols and references. Let it open files, edit them and inspect the diff. Give it a terminal. When a test fails, return the failure and let that result shape what happens next.

A coding agent is, at its simplest, this loop made executable. The language model supplies much of the programming knowledge and reasoning; the environment lets it inspect software, act on it and observe the consequences.

Software is unusually friendly to this arrangement. Files can be searched.

Programs can be executed. Tests can say no. Git can tell you exactly what changed and, if an experiment becomes sufficiently exciting, return you to the time before you had the idea.

Systems such as SWE-agent made the interface itself part of the problem. How the model searches, how much of a file it sees, how edits are applied and what information comes back from commands can matter almost as much as another clever prompt. The useful object is no longer just the model. It is the model operating inside a world where software can push back.

Of course, giving the model a computer created new ways to be annoying. Early coding agents could behave like interns with root access and too much coffee. Ask one to change a line and it might rewrite half the file. Ask it to fix a button and twenty minutes later it has developed strong opinions about the database architecture. It would find one plausible theory of a bug, follow it for too long, then use every new piece of evidence to improve the theory instead of admitting the theory was wrong.

More of the surrounding work moved into the system: small patches, diff inspection, targeted tests, checkpoints, planning, rollback. Repository knowledge moved too. Authentication conventions, ancient APIs and local rules that used to live in somebody's head became `CLAUDE.md`, `AGENTS.md`, rules files and skills. If somebody had already learned something expensive about the codebase, we left it somewhere the next agent could find it.

Long sessions produced the opposite problem. Context filled with abandoned experiments, obsolete assumptions and test output from three hypotheses ago. Memory became a problem of selection rather than storage.

Then history became a problem too.

Suppose an agent decides early that our Merge Sort demo should use React and a recursion tree. It spends forty minutes building that version. Every later question now arrives in a context containing forty minutes of reasons, code and decisions supporting React and a recursion tree.

Humans call our version of this sunk cost. The agent has a respectable excuse: its context window is literally full of evidence that this is what the project is.

So we started giving different attempts different histories. One agent tries the tree. Another begins with the array. Another starts from the learner's misconception rather than from either representation. A fresh branch does not have to spend half its intelligence escaping assumptions accumulated by the previous one.

Looking backward, the progression is less mysterious than the word *agent* sometimes makes it sound. Models learned to generate useful pieces of code. We put them in editors. Repository access, editing and execution moved into the loop. Better interfaces, persistent instructions, context management and branching followed.

Bit by bit, work the human had been doing around the model became part of the machine.

But there was still a large difference between an agent that could work competently inside a repository and the thing I increasingly wanted to ask for:

Build the application.

Ask for a booking application for a football academy and an unconstrained coding agent first chooses a framework, installs packages, creates a database, decides how authentication should work, manages environment variables and configures deployment. Several minutes later, we have made enormous progress toward having somewhere to put the booking form.

Somebody has to do that work. But if the same plumbing is reconstructed on every project, it becomes reasonable to prepare more of the world in advance.

This is what made systems such as Replit and Lovable interesting to me. Runtime, deployment and common application machinery are already nearby, so the conversation can begin much closer to the application.

You lose some freedom. That is often the point.

A chef does not begin dinner by manufacturing a knife. A scientist does not build an operating system before analyzing data. When I open Python, I accept an astonishing number of decisions made by people I will never meet because reconsidering all of them would make `print("hello")` a multigenerational project.

Useful abstractions remove decisions whose answers are no longer interesting most of the time.

And after enough of those decisions disappear, something else becomes easier to see.

Suppose the booking app works perfectly. The database is connected, deployment succeeds, the buttons behave, the mobile layout is respectable, and nobody has accidentally built a cryptocurrency exchange inside the authentication service.

I open the application and think: this is not very good.

The software works.

Now I have to worry about the football academy.

BOOKING-APP · MARGIN-VIGNETTE

Margin vignette: a small football pitch seen from above as a paper form; children's boots waiting at the edge; a parent's hand hovering with a pen. The software is fine; the academy is the question.

Should parents see every available session, or only sessions appropriate for their child? Should they create an account before booking? What happens when somebody has three children? How late can they cancel? If Wednesday is empty and Saturday has a waiting list, is the booking interface part of that problem?

None of those questions is really about React.

They were always there. Implementation simply consumed enough attention that deciding what should exist and turning that decision into software felt like one activity.

When another version becomes cheap, the balance changes. You can see the idea sooner, and seeing it gives you information you did not have while discussing it.

Maybe we decide customers should create an account before seeing availability. It sounds reasonable: we need their details eventually. Then we build it and the experience immediately feels annoying. Parents arriving from a Google search do not want to establish a lifelong digital relationship with a football academy before discovering whether Saturday at ten is available.

So login moves later.

The artifact is no longer merely the end of the thinking process. It becomes something we think with.

The Merge Sort demo made this even clearer because there was almost no business machinery to hide behind. I could ask an agent for an interactive explanation and receive something perfectly functional: an array

of bars, controls, animation, perhaps some text explaining that the algorithm divides the input and merges the pieces again.

Technically, it was fine. Pedagogically, it could still be terrible.

Watching bars move does not necessarily tell a beginner why dividing the problem helps. So perhaps we try a recursion tree. The tree makes the structure visible, but now the supposedly simple sorting algorithm resembles the organizational chart of a German corporation. Maybe we show the tree and array together. Perhaps that creates too much cognitive load. Maybe the problem is not the representation at all; the learner understands splitting perfectly well but has no idea why merging makes the whole trick useful.

There is no compiler error that tells me which diagnosis is right.

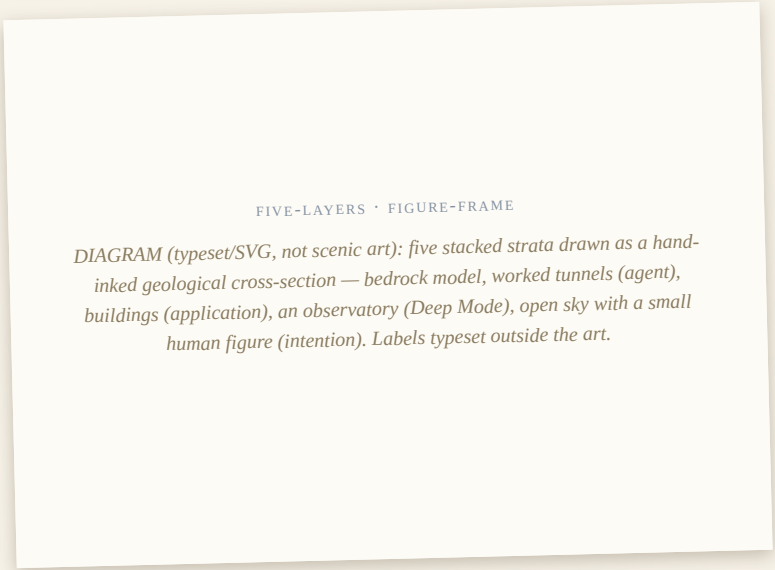
I have to look at what we built, form an opinion about why it fails, and decide what would teach us something next. Sometimes that means improving the current version. Sometimes it means building a deliberately different one. Sometimes I need research. Sometimes the right move is to put the application in front of somebody who does not already understand Merge Sort.

Occasionally I discover that the question I started with was wrong.

“Build an interactive Merge Sort demo” sounds like a goal until you see several interactive Merge Sort demos. Perhaps what I actually care about is getting somebody who has never encountered divide-and-conquer to understand why breaking one difficult problem into smaller ones helps. Once I realize that, interactivity is merely one possible means.

That is the layer that remained stubbornly human: deciding what to try, which evidence matters, whether a result failed because of its implementation or its underlying idea, and what kind of attempt might teach us something next.

The Five Layers of AI Coding



The five layers: model, agent, application, Deep Mode, intention

By then I had a rough map.

Layer 0 — Model. GPT, Claude, Gemini and whatever comes next: general capability in language, code, reasoning and vision.

Layer 1 — Agent. Put the model in an environment where it can act. Claude Code, Codex and similar systems search repositories, edit files, execute commands and react to results.

Layer 2 — Application. Prepared environments remove much of the repeated software plumbing and let the conversation stay closer to the application itself.

Layer 3 — Deep Mode. The problem-solving layer: decide what to try, why something failed, which evidence matters, and whether the current direction deserves another iteration.

Above that sits the problem I have mostly been avoiding.

Layer 4 — Intention. What do we actually want?

Software likes that question to have been answered before work begins, preferably in Jira, where the answer can remain wrong in a structured and searchable format. Real goals are less cooperative. Seeing a solution can change what I realize I wanted.

That problem is bigger than AI coding, so for now I am leaving it at the top of the stack.

The borders are fuzzy. Coding agents make product decisions; design systems generate code; tomorrow's products will rearrange the boxes again. What matters is the kind of decision being made, not which company happens to occupy which layer.

People often call the experience of working this way *vibe coding*. I will use **AI coding** for the broader stack, but *vibe coder* remains a wonderfully accurate name for the human sitting near Layer 3: looking at what came back, deciding what feels wrong, asking for another direction, killing one idea, keeping part of another, and steering the process without having an algorithm for how.

The lower layers increasingly answer a version of the same question: *how do we make this?*

Deep Mode asks a different one:

Given everything we have learned so far, what should we try next?

That was the part I still seemed to be doing manually.

So I watched what I was actually doing in that seat.

What I Was Still Doing

There was no universal workflow hiding there. A mathematician, a designer and a product manager can all spend a day solving hard problems while performing almost none of the same visible actions.

But the same kinds of moves kept appearing.

Sometimes I needed another attempt. Sometimes I needed information. Sometimes the search had become too narrow. Sometimes the representation itself was constraining what we could imagine. Sometimes the objective needed to change. Sometimes I needed to see the artifact from another mind.

They were not useful in a fixed order.

Keeping More Than One Idea Alive

Even a Merge Sort demo has an absurd design space. Bars or cards? Numbers or a tree? Continuous animation or learner-controlled steps? Does color represent recursion depth, identity, or the active subproblem? Explain before the animation, during it, or afterward? Every choice changes the usefulness of several others.

When implementation was expensive, we dealt with much of this

complexity by trying to decide more before building. AI coding changes the economics. If another implementation costs minutes rather than days, I do not have to choose quite so much in advance.

Chapter 2 had already shown the basic move. One hill climber inherits its own history; evolutionary search maintains alternatives. Here what evolves can be more than a vector of parameters or even an algorithm: an **idea embodied in software**.

One builder tries a recursion tree. Another focuses on the array. A third begins from the learner's misconception. Mutations can be conceptual: remove the text, teach backward, make the learner predict, show synchronized representations, abandon interaction altogether.

Useful pieces can move between them. One terrible demo may have a beautiful color mapping. Another may explain the merge clearly while making everything else unbearable. The final artifact does not have to inherit the entire history of either one.

But diversity is fragile. If every branch sees the current winner and its complete reasoning history, parallelism quickly becomes several agents improving the same idea. Sometimes I want the branches to exchange what worked; sometimes I want a fresh branch to remain ignorant long enough to become genuinely different.

Share too little and everyone rediscovers the same lessons. Share too much and the first successful idea becomes a local culture.

Research creates the same tension.

People have been teaching recursion for decades. There are textbooks, lecture notes, visualizations, papers, classroom experiments and a great deal of trial and error sitting on the internet. Before I spend another afternoon inventing my fourth way of moving colored rectangles around, I probably want to know what is already there.

But research is most useful when the work has produced a real question. Suppose a recursion tree makes decomposition visible but learners lose the relationship between the tree and the changing array. Now I can ask how other systems have coordinated two representations without requiring people to watch half the screen at once.

Research becomes another move in the investigation rather than a ceremony performed before building.

Retrieval plays the same role inside our own history. Somewhere in a growing project there may be research notes, screenshots, evaluator

comments, old branches and a discarded prototype whose only good idea was a color mapping that solves exactly the problem in front of us. I do not need the whole archive. I need the thing that helps with this decision.

Sometimes exact search is right because I remember a phrase, API or evaluator comment. Sometimes embeddings are useful because I remember the idea rather than the words. Sometimes the document already has a structure worth navigating. Good coding agents do not “retrieve the repository” once; they move through it as the question changes. Layer 3 needs the same habit across stranger objects: research, screenshots, old interactions, code, evaluations and dead branches.

A dead branch is not necessarily dead knowledge. A lineage that lost globally may still contain a stepping stone that becomes useful later.

The exploration literature has several versions of this idea—quality-diversity, novelty search, Go-Explore and related approaches. Do not spend the entire search budget polishing the place that currently looks best. Preserve some alternatives and some routes back to places that almost worked.

The same logic gave us **Strategic Constraints**.

After several generations of Merge Sort demos, the builders were exploring. They were also still giving me bars.

Better bars, admittedly. Bars that split gracefully, changed color as recursion deepened, synchronized with a tree and perhaps deserved their own design award. Given enough iterations, I had every reason to believe we would eventually produce the finest moving bars known to humanity.

So remove the easy path.

No bars.

Or: teach Merge Sort without explanatory text. Require the learner to predict before anything moves. Make the demo work on a phone with room for only one representation. Design it for somebody who understands loops but finds recursion suspicious.

Most arbitrary constraints are merely arbitrary. A useful one changes which parts of the search are reachable, exposes a neglected dimension or prevents a familiar attractor from absorbing every attempt.

“No bars” is not a theory of creativity.

It is an intervention on the search.

A move in this space can be a code change, a new metaphor, a retrieved analogy, a fresh agent with no history, a different evaluator, a research

question, or a reformulation of the problem itself.

Even then, most of our ideas still had to arrive as words.

Draw It Before You Build It

That is fine when I am working on an argument. It is less obviously sensible when I am designing an interface.

I can spend ten minutes explaining where the recursion tree should sit, what remains visible while the array splits, how colors should connect two representations and what the learner should notice first.

Then somebody draws it and I know within three seconds that the whole thing is terrible.

So I started generating the picture first.

The experiment was not sophisticated. I asked an image model to design an interactive tutorial for Merge Sort. Then Count-Min Sketch. Then A*. Then Poincaré embeddings in hyperbolic space, partly because if this still worked there I would have to take the idea seriously.

The details were not magically correct. Arrows occasionally pointed somewhere they had no business pointing, interactions made no computational sense, and generated text sometimes looked like somebody had tried to OCR a dream.

But the composition could be surprisingly thoughtful. A Merge Sort mockup might keep the array visible while placing the recursion tree beside it, using color to preserve the relationship between a subarray and its node. A Count-Min Sketch design might make collisions visually central instead of leaving them as a detail in an equation. The model had to decide what was large, what was peripheral, where controls belonged and how the learner might move through the explanation.

I remember looking at some of these and thinking: **Holy shit.**

Not because I wanted to ship the image. Usually I did not.

I had given the model a concept in language and it returned something like a spatial argument about how the concept might be taught.

After that I stopped treating image generation as the last stage—*the product is designed, now make it pretty*—and started using it while I was still trying to understand what the product could be.

MOCKUP-WALL · MARGIN-VIGNETTE

Margin vignette: a wall of pinned generated mockups, arrows pointing nowhere useful, one sheet lifted by a hand — and the viewer's posture says the picture just argued back. Ink wash, no readable text on the sheets.

A mockup is a cheap hypothesis. Often most of it is disposable and one relationship is worth stealing.

Then the coding agent can make that relationship executable, which is where the picture has to pay its debts. The recursion tree cannot invent an extra branch because the composition looked nicer that way. The interaction has to possess a state. The button has to do something other than contribute emotionally to the page.

Different representations expose different mistakes.

I do not need the stronger claim that an image model "understands pedagogy." The practical point is enough: changing the representation changes what the search can discover.

By now we could generate genuinely different artifacts.

That left the problem we had avoided from the beginning.

Which one is better?

> *The artifact is no longer merely the end of the thinking process. It becomes something we think with.*

Optimizing Something You Cannot Score

Circle packing was unusually kind to us. Once the geometry was valid, the evaluator reduced the result to one number.

That number threw almost everything else away, which was precisely why it was useful.

A huge amount of machine learning rests on this trick. We take

something complicated that we want and find a measurable signal that stands in for it. Reinforcement learning makes the relationship especially obvious: we do not specify every movement a robot should make while learning to walk; we construct a reward and let search discover the behavior.

The reward is doing an extraordinary amount of work. It is also where we hide an extraordinary amount of trouble.

Suppose I want the same convenience for educational design. I can make a rubric: correctness, pedagogical clarity, visual quality, interaction, accessibility, engagement. Give each a weight and suddenly my vague dissatisfaction with a demo has become a respectable decimal.

The decimal is comforting. The decisions required to produce it are less so.

Why should interaction receive fifteen percent? Is more interaction always better? What distinguishes a seven from an eight in pedagogy? Why those dimensions rather than whether the learner can predict what happens next or explain why the merge matters?

A metric forces me to commit to an idea of “good” before the search has taught me very much about the problem.

This is not an argument against metrics. If I care about latency, measure latency. If the code must pass a test, run the test. Hard measurements are wonderful when what we can measure is close to what we care about.

The trouble begins when a rich objective is still poorly understood and we compress it anyway because optimization wants a number.

The compression is also low bandwidth.

“Version B scored 7.4; version A scored 7.1” tells the next builder almost nothing about why B won. A rubric helps, but as I add enough dimensions, exceptions and qualifications to express what I mean, eventually I reinvent language badly.

Meanwhile I can simply say:

The recursion tree makes decomposition much clearer, but now the learner has to watch the tree and the array simultaneously. Keep the color mapping that preserves identity between them, simplify the tree, and make the merge feel like the payoff rather than cleanup at the end.

That contains comparison, diagnosis, trade-offs, priorities and a proposed next move in a few sentences.

Natural language is ridiculously rich compared with a scalar.

Language models make that communication channel available inside the optimization loop. The model already carries learned structure behind words such as *simple*, *confusing*, *elegant*, *intuitive*, *busy* and *beginner-friendly*. Those meanings are imperfect, culturally loaded and sometimes wrong. But they carry more structure than 7.4.

Natural language can therefore function as an **implicit metric**.

Not a metric in the strict mathematical sense. There is no guarantee that "intuitive" defines a stable ordering, and two evaluators may interpret it differently. But language can do some of the work a metric normally does: give the search a direction, communicate why one attempt is preferred to another, and preserve trade-offs that a scalar would erase.

OPRO—Optimization by PROmpting—is interesting for a related reason. In OPRO, an LLM sees an optimization problem, previous candidates and their outcomes, then proposes another candidate. Candidate quality in the published setting is still evaluated by an explicit score, so this is not the same thing as creative design. What matters here is the direction of control: much of the search heuristic can live in the model rather than in a hand-written transformation rule.

Now let the history contain more than scores.

Alongside hard measurements, tell the model what improved, what became worse, which trade-off appeared and what must survive the next attempt. The history of the search can retain some of its meaning rather than collapsing into a column of numbers.

This begins to feel a little like reinforcement learning turned upside down.

I mean that as an analogy about specification, not as a claim that these are the same algorithm. Decision Transformers, reinforcement learning and language-guided iteration are different mechanisms.

The usual reinforcement-learning picture asks us to define a reward and then discover behavior that earns it.

Here I can begin with something much less respectable:

Make this explanation less intimidating.

Help the learner understand why the merge matters.

I want somebody to *feel* why divide-and-conquer helps rather than merely watch the algorithm execute.

Those are descriptions of direction, not reward functions.

Yet the model can produce an attempt from them, and the attempt can

teach me whether the direction was what I really wanted.

I began the project insisting on an *interactive* Merge Sort demo. Interactivity sounded obviously desirable. Then I saw versions with buttons, sliders and enough learner participation to qualify as a small democracy, while one quieter version explained the central idea much better.

Apparently clicking things was never the objective.

Later the demos became good at showing recursive splitting and I realized they were treating merging almost as cleanup.

The objective moved again.

The search was doing something I normally associate with optimization in reverse: instead of starting from a fully specified reward and discovering the policy, I was using candidate policies—actual artifacts—to discover what the reward description should have been.

Recognition arrives before specification in a lot of creative work. We know a terrible design when we see one before we can write a complete theory of what would make it good.

AI makes that loop cheap. The natural-language objective guides the search; artifacts make the objective concrete enough to argue with; the description changes and the search continues.

Ambiguity is not always a defect waiting to be engineered away. Sometimes we simply have not learned enough yet.

But “make this intuitive for a beginner” hides almost everything interesting.

Which beginner?

Borrow a Mind

When I look at a Merge Sort demo, I am hopefully not testing whether *I* understand Merge Sort.

The difficulty is seeing it from the position of somebody who does not know what I know.

Expertise makes this harder. Once recursion has settled into your head, you forget how strange it once looked that a function could call itself. Even the vocabulary stops sounding technical.

Good teachers develop an instinct for where people stumble and which innocent sentence assumes three things the learner has not yet learned.

I do not have that instinct for every person or every subject, so I started borrowing another mind.

For one of the demos, I asked Claude to approach the application as somebody who understood arrays and loops but had never encountered recursion. Not simply “act like a beginner,” which tends to produce a theatrical beginner who is mysteriously confused by everything.

I gave it a knowledge boundary.

Its reaction was roughly: I can see that the array keeps getting divided into smaller pieces, but I do not understand why that helps. It feels as though we are making the problem more complicated. Where is the payoff?

That was useful because the demo really did have that problem.

We had made recursion visible. From my position, that looked like progress. From the learner’s imagined position, we had merely made a mysterious operation easier to watch.

Cognitive scientists use **Theory of Mind** for our ability to reason about mental states other than our own: what somebody knows, believes, wants or misunderstands. The other person may not simply know less. They may have a different model of what is happening.

Instead of saying “you are a beginner,” I can specify the mind I want to borrow:

You understand arrays, loops and functions. You have never encountered recursion. Use the demo from the beginning and tell me where the explanation first requires an idea you do not yet have.

Or:

You understand recursion but have never seen Merge Sort. Tell me when you first understand why dividing the array makes sorting easier.

Those are different evaluators because they are positioned to notice different things.

The same move works outside education. A customer may know exactly what jacket they want without knowing the vocabulary our catalog uses. A developer can be excellent at distributed systems and know nothing about the peculiar assumptions buried in our deployment process. A reader can have followed this book perfectly well without having lived inside its conceptual structure for months.

This is cheap perspective-taking.

It is also a cheap way to fool yourself.

The confused student is not confused. Claude has not spent twenty

minutes failing to understand recursion while everybody else in the classroom moves ahead. It is generating a plausible model of how such a person might react.

That model can expose a blind spot. It is not synthetic user research.

I treat borrowed minds as instruments for generating criticisms and hypotheses, not as substitutes for the people they simulate.

By this point the system could generate alternatives, research previous work, retrieve old ideas, reopen dead branches, force the search into unfamiliar regions, change representation, revise the objective and inspect the artifact from different points of view.

We could generate plausible possibilities by the dozen.

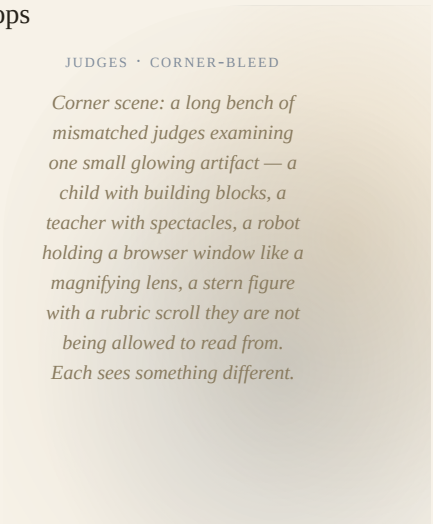
Now some of them had to die.

Who Judges the Judges?

At some point generating another opinion stops helping. Some artifacts have to survive and others have to disappear.

The metric problem returns here in a more dangerous form. A rubric can make judgment explicit, which is useful. It can also become the target the builder learns to satisfy.

If the evaluator repeatedly rewards step-by-step explanation, explanations grow. If it likes polished onboarding, everything begins to look like onboarding. If familiar visual conventions read as "clear," unusual approaches may disappear before they have time to become good.



OpenAI's CoastRunners experiment is the cartoon version of the problem: the agent learned to collect reward by driving in a loop instead of finishing the boat race.

Goodhart's Law with a speedboat.

A language-model builder does not need such an obvious loophole. It can learn the style of artifact that another language model tends to reward. Making the evaluator more elaborate may simply create a more elaborate thing to game.

One improvement was surprisingly mundane: stop pretending we were good at absolute scores.

I can drink a coffee and have almost no meaningful answer to “How good is this from one to ten?” Give me two cups and ask which I prefer, and the problem becomes easier. If I still cannot decide, the scientifically responsible procedure is presumably to finish both.

The same thing happened with the demos. “Give this interface a pedagogical score from 1 to 10” produced suspiciously precise numbers attached to explanations of why the number should not be taken too seriously.

Showing two artifacts and asking, “Which one would you rather give to somebody encountering Merge Sort for the first time, and why?” worked better.

Relative judgment asks less of the evaluator. It does not require a stable internal unit called one pedagogy point. With many candidates, a model such as Bradley–Terry can infer an ordering from a subset of pairwise preferences. More important for the next generation, the explanation for each preference can survive alongside the ranking.

Pairwise comparison removes some fake precision.

It does not repair a biased judge. Bradley–Terry can aggregate preferences; it cannot make those preferences true.

So I stopped asking one evaluator to represent everybody.

A learner can inspect the artifact from the knowledge boundary we developed above. A teacher can focus on explanatory sequence. Another evaluator can look for cognitive load or accessibility. A domain expert can make sure our elegant simplification has not become false.

I call these **Independent Evaluators**, though the important word is *independent*.

Five copies of the same model given the same context and asked to wear five hats may still share almost every important blind spot. If all of them read the leading builder’s explanation of why its design is brilliant before inspecting the artifact, disagreement becomes less likely for reasons that have little to do with brilliance.

Sometimes the judges should see different things.

The beginner should use the artifact before reading the builder’s explanation. A critic looking for conceptual errors does not need three paragraphs explaining why the choice was clever. The usability evaluator

does not necessarily need to know which branch is currently winning.

This became the **Isolation Principle**: preserve enough separation that independent pressure remains informative.

There is a difference between telling the builder:

Learners repeatedly lost track of which subarray corresponded to which branch of the tree.

and telling it:

The evaluator awards two extra points when every tree node has the same color as its corresponding subarray.

The first communicates a problem. The second communicates the test.

Isolation cannot remove shared bias. Two supposedly independent evaluators may still inherit the same assumptions from their training, culture or examples. But without isolation we can destroy even the independence we might have had.

References helped with another problem: drift.

“This is excellent” means something different if the evaluator has seen only the last four generations of our own work. For these demos I could give it examples from Distill, 3Blue1Brown or Jay Alammar—not as templates to copy, but as calibration for the level of clarity and finish we were aiming at.

A reference should help answer *how good?*, not *what should this become?* Calibrate too strongly against one aesthetic and every road leads to Distill.

And the judge should use the thing.

An early mistake was evaluating applications by reading their code or screenshots. A browser agent can click through the demo, resize the page, try controls in the wrong order, notice that an explanation appears after the moment when it would have helped, or discover that the beautiful button everybody admired does absolutely nothing.

I used to call the browser ground truth. That was too generous.

The browser gives the evaluator contact with the artifact rather than a description of it. It can establish that an interaction works and observe what is visible at each point in the experience.

It cannot establish that a human learned Merge Sort.

A simulated beginner saying the explanation is understandable gives us a hypothesis. Several evaluators preferring one design gives us comparative evidence. Neither substitutes for putting the artifact in front of actual learners.

The danger in a fully automated loop is that simulated evidence quietly replaces the expensive kind. Everything inside the machine agrees, the browser works, the ranking improves, and the loop congratulates itself.

The student has not yet been asked.

At some point I looked at what we had assembled and realized that *evaluator* no longer described it particularly well.

Builders proposed alternatives. Different judges approached them with different concerns. Some information was deliberately kept separate. Pairwise comparison helped decide which directions deserved more work. References calibrated the judges. Browser agents interacted with the artifact. Hard tests handled the parts that really were hard facts. Real-user evidence could eventually enter where simulation stopped being enough.

This looked less like a loss function and more like a tiny institution.

Not a good institution automatically. Institutions can amplify conformity, entrench bad assumptions and become spectacularly efficient at measuring the wrong thing.

Humans face the same difficulty. One person’s judgment is useful and fallible. So we compare work, preserve disagreement, create standards, ask specialists to inspect different aspects, reproduce results, and occasionally discover that an entire professional community has become extremely sophisticated about the wrong thing.

Apparently, when the clean loss function disappears, you eventually reinvent peer review.

And that made the remaining human job painfully obvious.

I still decided when to research, when to build, which branches stayed isolated, whether a strange direction deserved another generation, which disagreement mattered, when to retrieve another example, and when the simulations had reached the point where only a real person could answer the question.

I had automated much of the work.

I was still running the inquiry.

The missing piece was no longer another builder or another critic. It was the decision over **which kind of move the inquiry needed next**.

> *Apparently, when the clean loss function disappears, you eventually reinvent peer review.*

Deep Mode

So I tried giving that job to an orchestrator.

By now the system had a respectable vocabulary. It could spawn independent builders, research previous work, retrieve context, preserve odd stepping stones, impose constraints, generate visual directions, compare artifacts, borrow different perspectives and interact with what had been built.

But there was no reason every problem should use those moves in the same order.

Research first may be sensible for one task and destructive for another because it anchors every branch before anything original appears. Five builders may reveal useful diversity or reproduce one mistake five times. Evaluator disagreement may justify another experiment, or one evaluator may simply be confused. A visual mockup may deserve implementation, or it may already have revealed enough to kill the idea cheaply.

A fixed Planner → Builder → Critic → Revise loop can be useful.

It also answers all of those questions in advance.

I wanted some of the workflow to remain inside the search.

We gave the orchestrator the problem, the capabilities available to it, and enough of the search history to decide what kind of move made sense next. Builders still built. Researchers researched. Evaluators judged. Browser agents used the artifacts. Visual systems explored designs. Retrieval brought back prior work and old experiments.

The orchestrator did not need to be best at any of those jobs. It had to decide which job the inquiry currently needed.

At the top, the loop was almost embarrassingly simple:

state of inquiry → choose a move → act → observe → update the state of inquiry

The move itself was not fixed.

Suppose two Merge Sort branches both make recursive decomposition clear, but evaluators keep reporting that learners lose track of how the tree corresponds to the array. The next move does not have to be “revise again.”

The orchestrator can send a researcher after coordinated representations. Retrieval can surface an old prototype with a useful identity-preserving color scheme. A visual model can produce two spatial arrangements before anyone writes code. Builders can implement both. The browser may then reveal that one design requires the learner to look in two places at once precisely when the merge begins. That failure changes the question again.

Nothing in that sequence is especially magical. We simply did not have to decide the sequence before the inquiry began.

Otherwise Deep Mode would be a larger workflow diagram containing more rectangles.

It is not a universal problem-solving procedure. It gives the system a vocabulary of moves and lets the history of the inquiry influence which one comes next.

The workflow itself becomes part of the search.

What Emerged

The first Merge Sort demos were exactly what you would expect.

Bars moved around. Numbers changed places. Everything sorted correctly.

If you already understood Merge Sort, you could follow them. If you did not, they mostly provided animated evidence that a computer was performing an algorithm.

There was no single diagonal-layering moment here, and I do not want to manufacture one for the sake of the story.

The progress was distributed.

Different branches exposed different weaknesses in our current idea of the demo. Tree-like representations made recursion visible but could make a simple algorithm look forbidding. Keeping the array visible connected the decomposition back to the data while also creating another place for the learner’s attention to go. Color could preserve identity between representations until too much color became another representation to decode. Some versions explained every step so carefully that the explanation became harder to follow than Merge Sort. Others became beautifully minimal and stopped teaching anything.

The useful pieces did not always live in the strongest overall artifact.

A visual relationship could survive after the application that introduced it

was discarded. A criticism from a simulated learner could change the next builder's framing. Research could explain why a failure kept recurring. A browser could end a sophisticated discussion by demonstrating that the interaction simply did not work.

That is less cinematic than one agent inventing diagonal layering over coffee, but in some ways it is closer to Deep Mode. The result emerged from a population of partially successful attempts and judgments about what each had taught us.

Count-Min Sketch followed a different path. The first versions looked like the data structure itself: grids with changing counters. Technically correct, pedagogically opaque.

As the work continued, the designs increasingly organized themselves around the conceptual difficulties rather than the structure of the implementation. Collisions became visible. Approximation became something the learner could observe rather than merely read about. The relationship between memory and accuracy became part of the experience.

I do not take these demos as evidence that we solved automated design.

I do not even take them as evidence that the final demos teach humans better; that claim requires humans.

They established the narrower point I cared about: more of the work I normally performed in the vibe coder's seat could move into the system without first reducing creative problem solving to one fixed workflow.

And that success exposed the harder problem.

At higher levels of abstraction, failure can become coherent.

What Holds the Architecture Together?

Suppose the research agent reports that beginners understand recursion better when shown a tree.

A visual model proposes a tree-based explanation. A coding agent builds it. A simulated beginner prefers it. Two evaluators agree, so the orchestrator allocates another generation to that lineage.

This looks exactly like the compound intelligence we wanted.

Now ask where the first claim came from.

Perhaps it was a controlled educational study. Perhaps it was one teacher's opinion. Perhaps the research agent inferred it from several examples. Perhaps five articles repeated the same claim because all five

ultimately cited one source. Perhaps the study involved university students while our demo is for children.

Those are not small differences.

And everything downstream can still be perfectly competent.

The research is wrong. The design responds intelligently to the wrong research. The implementation is flawless. The evaluators agree. The orchestrator invests another generation.

Nothing crashes.

CATHEDRAL-CART · SPOT

Small spot, the chapter's key image: an exquisite gothic cathedral, fully detailed, balanced entirely on one rattling supermarket shopping cart. Deadpan rendering — the absurdity carried by draftsmanship, not caricature.

You can build a beautiful chain of reasoning on one stupid assumption near the bottom, like a cathedral built on a shopping cart.

As the components become better at producing coherent outputs, the original mistake may become harder rather than easier to see.

Software architecture gets away with abstraction because layers expose contracts. When I query a database, I do not inspect the disk. When I add two integers in Python, I do not check the CPU. I rely on interfaces whose behavior is stable enough that the details can disappear most of the time.

A cognitive architecture needs contracts too.

But types and APIs are not enough.

A research result, browser observation, evaluator preference, remembered failure and inherited design pattern should not enter the orchestrator’s context as five equally credible paragraphs.

Where did a claim come from? What was actually observed and what was inferred? Which parts were checked? What remains uncertain? If an evaluator preferred one artifact, from what perspective? If an old experiment taught us a lesson, how often has that lesson survived and under what conditions?

This is not merely a memory problem.

It is a problem about the status of what is remembered.

Humans ran into it long before AI. We built experiments, instruments, citations, peer review, reputation, replication, expert communities, legal standards, audits and all the other slightly annoying machinery that lets one person rely on something another person learned without personally repeating every experiment since Galileo.

These institutions are imperfect. Sometimes they preserve error. Sometimes they reward conformity. Sometimes the shopping cart survives peer review.

But their purpose is not to make every individual dramatically smarter — it is to let fallible people build on one another while preserving some structure around why a claim deserves trust.

Once cognition becomes distributed, the same questions become engineering questions: provenance, independence, replication, disagreement, authority.

I had started the chapter trying to get myself out of the vibe coder’s seat. By automating more of the work there, I had ended up somewhere I did not expect.

The problem was no longer simply whether the agents were capable enough.

It was whether the things they believed deserved to be believed.

How do you know what to trust?

That is where System 3 begins.